TRABALHO PRÁTICO

Classificação Multiclasse com Redes Neurais

Dry Bean Dataset

1. Visão geral do trabalho

Neste trabalho você irá treinar uma rede neural do tipo MLP (Multilayer Perceptron) para classificar sete variedades de feijão a partir de características morfológicas extraídas de imagens dos grãos.

O objetivo não é apenas obter um modelo com boa acurácia. Você deve explorar o problema em etapas, tomar decisões fundamentadas e comparar diferentes configurações da rede. O relatório é parte central da avaliação.

2. O dataset

O Dry Bean Dataset foi escolhido porque concentra vários desafios reais em um só problema:

- Features com escalas muito diferentes.
- Distribuições assimétricas com cauda longa, que podem prejudicar o treinamento.
- Features altamente correlacionadas entre si, podendo gerar redundância de informação.
- Classes com quantidades muito diferentes de amostras.
- Com 13.611 amostras, é grande o suficiente para resultados robustos.

2.1 Origem e download

O Dry Bean Dataset foi criado por Murat Koklu e Ilker Ali Ozkan (2020) e está disponível no UCI Machine Learning Repository: https://archive.ics.uci.edu/dataset/602/dry+bean+dataset (ou https://archive.ics.uci.edu/static/public/602/dry+bean+dataset.zip)

O arquivo principal é um Excel (.xlsx). Você pode baixá-lo manualmente ou diretamente pelo código usando requests e zipfile. É importante consultar a documentação do UCI para entender como o dataset foi coletado antes de começar a análise.

2.2 Classes-alvo

A coluna Class contém o rótulo de cada amostra. Há sete variedades:

Classe	Nome
BARBUNYA	Feijao Barbunya
BOMBAY	Feijao Bombay
CALI	Feijao Cali

Classe	Nome
DERMASON	Feijao Dermason
HOROZ	Feijao Horoz
SEKER	Feijao Seker
SIRA	Feijao Sira

2.3 Descrição das features

O dataset possui 16 features de entrada, todas numéricas e contínuas, derivadas da análise de forma e morfologia dos grãos:

Feature	Descrição
Area	Área da região do grão (pixels)
Perimeter	Comprimento do contorno do grão
MajorAxisLength	Comprimento do eixo maior da elipse equivalente
MinorAxisLength	Comprimento do eixo menor da elipse equivalente
AspectRation	Razão entre eixo maior e menor
Eccentricity	Excentricidade da elipse equivalente
ConvexArea	Area do menor polígono convexo que contém o grão
EquivDiameter	Diâmetro de um círculo de mesma área
Extent	Razão entre área do grão e área da bounding box
Solidity	Razão entre área do grão e ConvexArea
Roundness	Circularidade calculada sem eixos elípticos
Compactness	Medida de compacidade da forma
ShapeFactor1-4	Quatro índices de forma derivados das demais medidas
Class	Tipo do feijão (7 classes — variável alvo/rótulo)

3. Etapas do trabalho

O trabalho é dividido em quatro etapas. Você deve entregar um único Jupyter Notebook organizado com títulos. O notebook deve estar totalmente executado — todas as saídas visíveis.

Etapa 1 — Análise Exploratória de Dados (AED)

Antes de construir qualquer modelo, você precisa entender os dados. A AED guia todas as decisões de pré-processamento e seleção de features que virão a seguir.

1.1 Inspeção inicial

Carregue o dataset e verifique sua estrutura básica: número de linhas e colunas, tipos de dados, presença de valores ausentes e contagem de amostras por classe.

Questões para o relatório

1. Quantas linhas e colunas o dataset possui? E o esperado conforme a documentação do UCI?
2. Ha valores ausentes? De que tipo são as features?
3. Existem classes com muito menos amostras? O que isso pode causar no treinamento? Como a acurácia pode nos enganar nesse caso?

1.2 Verificação e remoção de duplicatas

Verifique se existem registros duplicados no dataset.

Questões para o relatório

4. Quantos registros duplicados foram encontrados? Qual seria o impacto de mantê-los?

1.3 Distribuição das features

Features com distribuição assimétrica e cauda longa podem prejudicar o treinamento porque os valores extremos (outliers) dominam o gradiente. Plote um histograma para cada feature numérica e observe a forma das distribuições.

Transformação logarítmica — como aplicar

Uma transformação logarítmica pode atenuar o efeito dos valores extremos. Pesquise o conceito de skewness (assimetria) e verifique quais features são candidatas a essa transformação. Você pode aplicar np.log1p(x) para a transformação logarítmica. Aplique ANTES da normalização e APENAS nas features com assimetria acentuada. Plote os histogramas antes e depois para visualizar o efeito.

Questões para o relatório

5. Quais features apresentam distribuição assimétrica com cauda longa?
6. A transformação log alterou visivelmente a distribuição? Como isso pode beneficiar o treinamento?

1.4 Análise de correlação de Pearson

Correlação mede o quanto duas variáveis caminham juntas linearmente. Features altamente correlacionadas carregam informação redundante — incluir ambas na rede pode não trazer ganho e dificultar o aprendizado.

Correlação de Pearson auxilia nessa verificação. O resultado varia entre -1.0 e +1.0:

- +1.0: quando uma sobe, a outra sobe proporcionalmente (correlação positiva).
- -1.0: quando uma sobe, a outra desce proporcionalmente (correlação negativa).
- 0: nenhuma relação linear entre as duas variáveis.

Calcule a matriz de correlação com df.corr() e visualize com um heatmap (seaborn.heatmap). Valores próximos de +1.0 indicam alta correlação positiva; próximos de -1.0, negativa; próximos de 0, pouca relação linear.

Questões para o relatório

7. Identifique pelo menos 3 pares de features com correlação acima de 0.90.
8. O que é multicolinearidade? Features muito correlacionadas trazem informação nova ou são redundantes? Features muito correlacionadas são obrigadas a serem removidas do dataset de treinamento? É possível que ao não removê-las ainda sim obtenhamos bons resultados?

1.5 Boxplots por classe

Um boxplot mostra como os valores de uma feature se distribuem para cada grupo (aqui, cada classe de feijão). Ele exibe: valor mínimo, primeiro quartil (Q1), mediana, terceiro quartil (Q3) e valor máximo. Valores fora dos limites aparecem como pontos isolados (outliers/extremos).

Para gerar um boxplot por classe, use df.boxplot(column='NomeFeature', by='Class'). Gere boxplots para pelo menos 4 features. Sugestão: Area, Eccentricity, Roundness e ShapeFactor1 — elas representam quatro tipos de informação diferentes (tamanho, forma elíptica, circularidade e índice composto) e tem baixa correlação entre si.

Visualize como cada feature se distribui para cada classe. Features que separam bem as classes visualmente tendem a ser mais informativas para o modelo.

Questões para o relatório

9. Com base no heatmap e nos boxplots, quais features você selecionaria para treinar o modelo?
10. Quais features melhor separam as classes visualmente?
11. Quais classes apresentam maior sobreposição? Você espera que o modelo erre mais nessas classes?

Etapa 2 — Pré-processamento

2.1 Codificação da variável alvo

A coluna Class contém texto. Use LabelEncoder do scikit-learn para converter os nomes das classes em inteiros (BARBUNYA=0, BOMBAY=1, ..., SIRA=6). O inteiro gerado é apenas um identificador — não representa uma ordem ou distância entre as classes.

A função de perda sparse_categorical_crossentropy, que você já conhece dos trabalhos anteriores, aceita esses inteiros diretamente e trata cada valor como uma categoria independente.

2.2 Normalização das features

As features possuem escalas muito diferentes. Sem normalização, a rede daria muito mais peso as features com valores maiores simplesmente por causa da sua escala numérica — não por serem mais informativas.

StandardScaler vs MinMaxScaler — qual usar e por que

MinMaxScaler: comprime todos os valores para o intervalo [0, 1].

Problema: um único valor extremo 'estica' a escala inteira. Se Area tiver um valor de 100.000 enquanto a maioria fica em torno de 50.000, o 50.000 vira ~0.5 e os menores ficam comprimidos numa faixa pequena — perdendo variância útil para o aprendizado.

StandardScaler: subtrai a média e divide pelo desvio padrão. Resultado: média=0, desvio=1.

Vantagem: valores extremos ainda existem, mas não distorcem a escala dos outros.

Vantagem adicional: ReLU não tem limite superior — o StandardScaler se encaixa melhor porque não cria um teto artificial em 1.0 como o MinMaxScaler.

Para este dataset, com features assimétricas e valores extremos, StandardScaler é preferido.

Experimento sugerido: substitua por MinMaxScaler e compare as curvas de aprendizado.

Questões para o relatório

12. Por que não podemos ajustar o scaler nos dados de teste?

13. O que aconteceria com as predições em produção se o scaler usado no treinamento fosse diferente do aplicado nas novas entradas?

2.3 Tratamento do desbalanceamento com class_weight

Quando há classes com muito menos amostras, a rede tende a ignorar as classes raras durante o treinamento. Uma forma de corrigir isso sem alterar os dados e usar pesos de classe: erros nas classes raras recebem penalização maior no cálculo da loss.

O scikit-learn oferece compute_class_weight com o parâmetro class_weight='balanced' para calcular os pesos automaticamente. Esses pesos são passados para o método model.fit() pelo parâmetro class_weight.

Questões para o relatório

14. A classe BOMBAY recebeu o maior ou menor peso? Por que?
15. O que acontece quando o modelo erra uma amostra de BOMBAY vs uma de DERMASON com esses pesos?

2.4 Divisão treino / validação / teste

Divida os dados em três conjuntos: treino (usado para ajustar os pesos), validação (usado para monitorar o treinamento e ajustar hiperparametros) e teste (usado apenas para a avaliação final).

Use train_test_split com stratify=y para garantir que cada split mantenha a proporção de cada classe.

Questões para o relatório

16. Por que o parâmetro stratify e importante neste dataset especificamente?
17. Por que os dados de teste só devem ser usados na avaliação final, e não para ajustar a arquitetura ou os hiperparametros?

Etapa 3 — Construção, treinamento e experimentos

Esta é a etapa central do trabalho. Você irá construir e treinar múltiplas versões da rede, variando a arquitetura e os hiper parâmetros, e comparar os resultados de forma sistemática.

3.1 Conceitos necessários

Dropout

Dropout é uma técnica de regularização. Durante o treinamento, desativa aleatoriamente uma fração dos neurônios a cada passo, forçando a rede a aprender representações mais robustas. A taxa indica a fração desativada: Dropout(0.3) desliga 30% dos neurônios a cada passo.

IMPORTANTE: o Dropout é desativado automaticamente durante validação e teste.
Por isso, nas primeiras épocas, val_accuracy pode ser MAIOR que train_accuracy.
Isso acontece porque durante o treino a rede opera 'enfraquecida' (com neurônios desligados), enquanto na validação ela usa toda a sua capacidade.
Esse comportamento é esperado e correto — não é overfitting.

Você irá comparar um modelo sem Dropout (Exp. A) e com Dropout (Exp. A2) para ver o efeito nas curvas de aprendizado e no resultado final.

EarlyStopping

EarlyStopping interrompe o treinamento quando a métrica monitorada (val_loss) para de melhorar por um número definido de épocas consecutivas (parâmetro patience).
O parâmetro restore_best_weights=True restaura os pesos do melhor momento ao final — não os da última época, que pode já ter piorado.
Isso evita ter que adivinhar o número ideal de épocas e protege contra o overfitting.
Use: keras.callbacks.EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

Inicialização dos pesos — kernel_initializer

Os pesos iniciais podem influenciar a velocidade de convergência e a qualidade do resultado.
He normal (he_normal): recomendado para camadas com ReLU. Calibra a variância dos pesos com base no número de entradas, evitando gradientes que explodem ou desaparecem.
RandomNormal(mean=0.0, stddev=0.05): inicializa com valores aleatórios próximos de zero. Pode convergir mais lentamente porque os valores iniciais são pequenos.
No Experimento D você comparará os dois: observe as curvas de loss das primeiras épocas.
Importe com: from tensorflow.keras import initializers
Use: kernel_initializer='he_normal' ou kernel_initializer=initializers.RandomNormal(0.0, 0.05)

3.2 Arquitetura da rede

Você deve definir a arquitetura da rede. Use o seguinte como ponto de partida, mas modifique e justifique suas escolhas:

- Camada de entrada: shape igual ao número de features utilizadas.
- Camadas ocultas: Dense com ativação ReLU. Teste pelo menos duas configurações de número de camadas e neurônios.
- Dropout após cada camada oculta. Teste pelo menos duas taxas diferentes.
- Camada de saída: Dense com softmax e units igual ao número de classes (7).

Na compilação do modelo, use:

- loss='sparse_categorical_crossentropy' — compatível com rótulos inteiros (variável target).
- optimizer='adam' como ponto de partida. Você pode experimentar outros (sgd, rmsprop).
- metrics=['accuracy']

Questões para o relatório

18. Por que a camada de saída usa softmax com 7 neurônios?
19. Por que a função de perda é sparse_categorical_crossentropy e não binary_crossentropy?
20. O Dropout é aplicado na camada de saída?

3.3 Experimentos

Você deve realizar no mínimo os experimentos listados abaixo. Para cada um, registre os resultados na tabela comparativa e analise as diferenças no relatório.

Dica prática — como organizar os experimentos

Passo 1: implemente o pipeline completo para um único modelo (carregamento, pré-processamento, treinamento, avaliação) e verifique que tudo funciona corretamente do início ao fim.

Passo 2: só então generalize. Encapsule o treinamento em uma função que receba os hiperparametros como argumento (por exemplo, um dicionário com 'units', 'dropout', 'initializer', 'batch_size', 'use_class_weight', etc.).

Com essa função, rodar um novo experimento e apenas chamar a função com um dicionário diferente — sem copiar e colar código.

Tabela guia para preenchimento e análise/interpretação do grupo sobre os resultados obtidos:

Exp.	O que testar	Features	Acurácia	F1-macro	Épocas	Observações
A	Configuração base (referência)	Todas (16)				
B	Adicionar class_weight	Todas (16)				
C	Rede maior (3 camadas ocultas)	Todas (16)				
D	Inicializador diferente	Todas (16)				
E	Features selecionadas na AED	Selecionadas				
F	Minha melhor configuração	A definir				

Descrição de cada experimento:

Experimento A — Sem Dropout (referência):

Implemente a rede sem nenhuma camada de Dropout. Este experimento serve como baseline e permite visualizar o overfitting sem regularização.

Experimento A2 — Com Dropout:

Adicione Dropout após cada camada oculta (taxa sugerida: 0.3 na primeira camada e 0.2 na segunda). Compare as curvas de aprendizado com o Experimento A. O Dropout reduziu o overfitting? A acurácia final melhorou ou piorou?

Experimento B — class_weight:

Repita o Experimento A2 adicionando pesos de classe. Observe se o F1-score das classes minoritárias (BOMBAY, BARBUNYA) melhora. Verifique também se as classes majoritárias pioraram.

Experimento C — Arquitetura maior:
Adicione uma terceira camada oculta ou aumente o número de neurônios. As curvas mostram mais overfitting? O resultado melhora?

Experimento D — Inicializador diferente:
Compare he_normal com RandomNormal(mean=0.0, stddev=0.05). Observe as curvas de loss das primeiras épocas: qual converge mais rápido?

Experimento E — Features selecionadas na AED:
Use apenas as features escolhidas na seção 1.4. Remover features correlacionadas melhorou, piorou ou não alterou o resultado? Quais classes foram mais afetadas?

Experimento F — Sua melhor configuração:
Com base nos resultados anteriores, defina a configuração que você acredita ser a melhor: seleção de features, profundidade da rede, dropout, inicializador, class_weight, learning rate e batch size. Justifique cada escolha no relatório. Este é o modelo que você salvará.

3.4 Visualização das curvas de aprendizado

Para cada experimento, plote as curvas de loss e acurácia para treino e validação ao longo das épocas. As curvas são fundamentais para diagnosticar o comportamento do treinamento.

Como diagnosticar o treinamento pelas curvas

Como diagnosticar o treinamento pelas curvas

Bom ajuste: as curvas de treino e validação caem juntas e estabilizam em valores próximos.
Overfitting: a loss de treino continua caindo, mas val_loss sobe ou estabiliza em valor alto.
-> Soluções: aumentar Dropout, reduzir a arquitetura/parâmetros, usar mais dados.
Underfitting: ambas as curvas estabilizam em valores altos sem convergir.
-> Soluções: aumentar a arquitetura/capacidade da rede, treinar mais épocas, revisar o pré-processamento.
Lembre-se: nas primeiras épocas com Dropout, val_accuracy > train_accuracy e normal.

3.5 Salvando o melhor modelo

Use ModelCheckpoint para salvar automaticamente o melhor modelo durante o treinamento:

- monitor='val_loss' — salva quando a val_loss melhora.
- save_best_only=True — mantém apenas o melhor checkpoint.
- O arquivo salvo pode ser recarregado com keras.models.load_model().

Salve também o scaler e o LabelEncoder usando joblib.dump(). Esses três arquivos (modelo, scaler, encoder) são necessários para fazer predições em novos dados.

Questões para o relatório

21. Por que é necessário salvar o scaler junto com o modelo?
22. O que aconteceria se você aplicasse um scaler diferente na hora de fazer predições em produção versus o scaler usado no treinamento?

4 — Avaliação e análise dos resultados

4.1 Métricas de avaliação

Para cada experimento, calcule no conjunto de teste:

- Acurácia geral.
- Relatório completo com precision, recall e F1-score por classe (classification_report do scikit-learn).
- F1-score macro — média não ponderada do F1 por classe, que trata todas as classes igualmente independentemente de seu tamanho.

Como interpretar o classification_report

O relatório apresenta três métricas para cada classe:

Precision: de todas as amostras que o modelo classificou como classe X, qual fracao era realmente da classe X? Precision alta = poucos falsos positivos.
Ex.: precision=0.93 para BARBUNYA significa que 93% das vezes que o modelo disse 'e BARBUNYA', estava certo.

Recall: de todas as amostras que realmente sao da classe X, qual fracao o modelo acertou? Recall alto = poucos falsos negativos.
Ex.: recall=0.90 para BARBUNYA significa que o modelo encontrou 90% dos grãos de BARBUNYA.

F1-score: média harmônica entre precision e recall. Útil quando há desbalanceamento de classes.
F1 = 2 * (precision * recall) / (precision + recall)
Um F1 alto só e possível quando AMBOS precision e recall são altos.

F1-macro: média do F1 de todas as classes, sem ponderar pelo tamanho de cada classe.
Trata todas as classes igualmente — ideal para avaliar desempenho com desbalanceamento.

4.2 Matriz de confusão

Gere a matriz de confusão para o seu melhor modelo com confusion_matrix e visualize com seaborn.heatmap.

Como ler a matriz de confusão

Cada linha representa a classe REAL; cada coluna representa a classe PREVISTA.
Os valores na diagonal principal são as predições corretas.
Valores fora da diagonal são erros: ex., linha SIRA / coluna DERMASON mostra quantas amostras de SIRA foram previstas erroneamente como DERMASON.

Como usar para confirmar suas hipóteses da AED:
Na seção 1.5 você anotou quais pares de classes achava que seriam confundidos.
Agora verifique: esses pares aparecem com valores altos fora da diagonal?
Se sim, sua análise dos boxplots foi certeira.
Se não, pense no que o modelo capturou que você não havia percebido.

Como detectar se o desbalanceamento está afetando o aprendizado

Sinal 1: F1-score da classe minoritária muito abaixo da média geral.
Ex.: F1-macro = 0.92, mas F1 de BOMBAY = 0.60 — o modelo está ignorando BOMBAY.

Sinal 2: Recall baixo com precision alta para a classe minoritária.
O modelo raramente prevê aquela classe (prefere não arriscar) mas quando prevê, acerta.

Sinal 3: Na matriz de confusão, quase todas as amostras da classe minoritária aparecem classificadas como outra classe — a linha correspondente tem zeros na diagonal.

Se você identificar qualquer um destes sinais: teste o Experimento B (class_weight) e compare o F1 por classe antes e depois. Discuta os resultados no relatório.

Questões para o relatório

23. Qual classe obteve o menor F1-score? Isso era esperado com base nos boxplots da AED?
24. Quais classes o modelo confunde com mais frequência? Elas apresentavam sobreposição nos boxplots?
25. A classe BOMBAY foi bem classificada? O uso de class_weight alterou seus resultados?

26. Qual classe obteve o menor F1-score? Isso era esperado com base nos boxplots da AED?
27. Quais classes o modelo confunde com mais frequência? Elas apresentavam sobreposição nos boxplots?
28. Há sinais de que o desbalanceamento afetou o aprendizado? O class_weight (Exp. B) corrigiu?
29. Qual experimento produziu o melhor F1-macro? Foi o que você esperava antes de rodar?
30. O Dropout (comparando A vs A2) reduziu o overfitting? Como as curvas evidenciam isso?

31. Remover features correlacionadas (Exp. E) trouxe ganho real? Quais classes foram mais afetadas?
32. Justifique a configuração escolhida no Experimento F como sua melhor versão.

4.3 Análise comparativa dos experimentos

Com a tabela de experimentos preenchida, análise os resultados de forma crítica no relatório:

Questões para o relatório

33. Qual experimento produziu o melhor F1-macro? Foi o que você esperava antes de rodar?
34. O Dropout alto (se testado) melhorou ou piorou o resultado? Como isso se reflete nas curvas?
35. Remover features correlacionadas (Experimento E) trouxe ganho real? Quais classes foram mais afetadas pela redução de features?
36. O class_weight ajudou as classes minoritárias sem prejudicar as demais?
37. Qual inicializador convergiu mais rápido nas primeiras épocas?
38. Justifique a configuração escolhida no Experimento F como sua melhor versão.

5. Formato de entrega

Entregue um único arquivo .zip contendo:

- notebook.ipynb — Jupyter Notebook com todo o código organizado por seções. O notebook deve estar totalmente executado (todas as saídas visíveis).
- relatorio.pdf — Deve responder às questões de cada etapa, apresentar as figuras geradas e a tabela comparativa de experimentos preenchida.
- melhor_modelo.keras — Arquivo do modelo salvo pelo ModelCheckpoint.
- scaler_final.pkl e label_encoder.pkl — Salvos com joblib.
- Não é necessário incluir o dataset.
- **O trabalho deve ser realizado em dupla ou em trio.**

6. Desafio extra — Deploy no Streamlit

Se você quiser ir além, implemente uma aplicação web simples usando Streamlit para servir o modelo treinado. A aplicação deve:

- Carregar o modelo salvo (.keras), o scaler (.pkl) e o encoder (.pkl).
- Apresentar campos de entrada para cada uma das 16 features (ou apenas as selecionadas, se for o Modelo E ou F com features reduzidas).
- Exibir a classe prevista e as probabilidades de cada classe em um gráfico de barras.

Para executar localmente: instale o Streamlit com pip install streamlit e rode com streamlit run app.py.

Dica para o Streamlit

Use st.sidebar.number_input() para cada feature de entrada.
Aplique o mesmo pré-processamento do treinamento: se usou log1p, aplique antes do scaler.
Use scaler.transform() — nunca scaler.fit_transform() — na entrada do usuário.
model.predict() retorna as probabilidades de cada classe; use .argmax() para obter a classe prevista
e le.inverse_transform() para converter o indice de volta ao nome da classe.

Observação: esse desafio não vale nota extra para o presente trabalho. Contudo, ele irá garantir um benefício na avaliação da P2 ou P3.